

ECOSYSTEM SURVIVAL



- Cristian Castañeda
- Julian Carvajal
- Juan Pablo Angulo

GAME CONCEPT AND GOAL

OBJECTIVE: REACH SAFE ZONES (X)



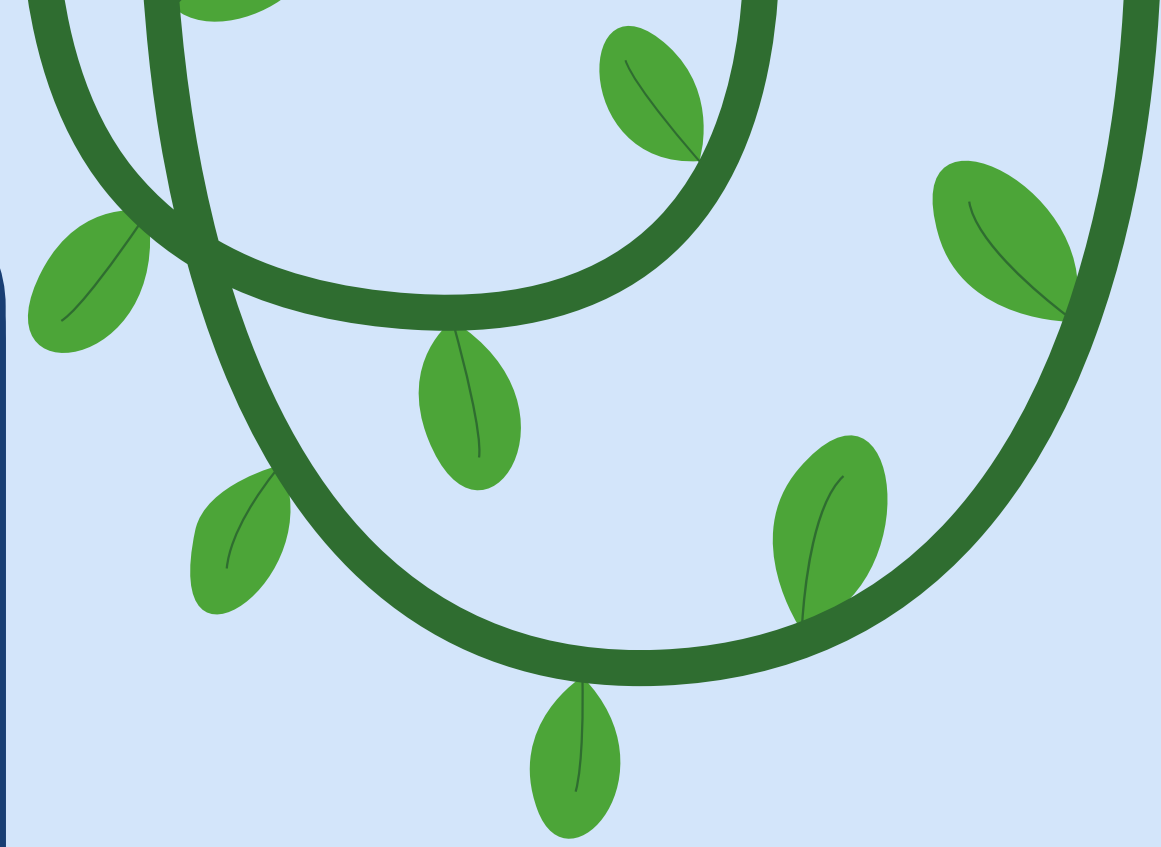
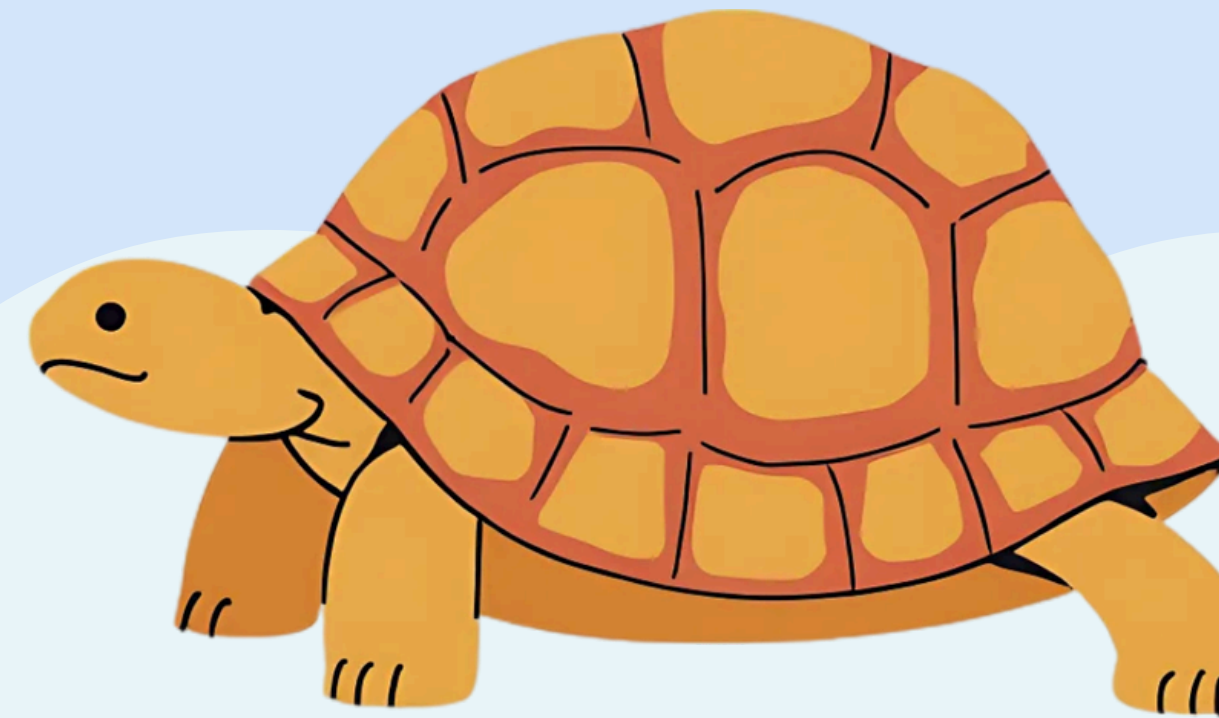
FOREST ECOSYSTEM SURVIVAL

LOSS CONDITION: SPECIES EXTINCTION (POPULATION = 0)



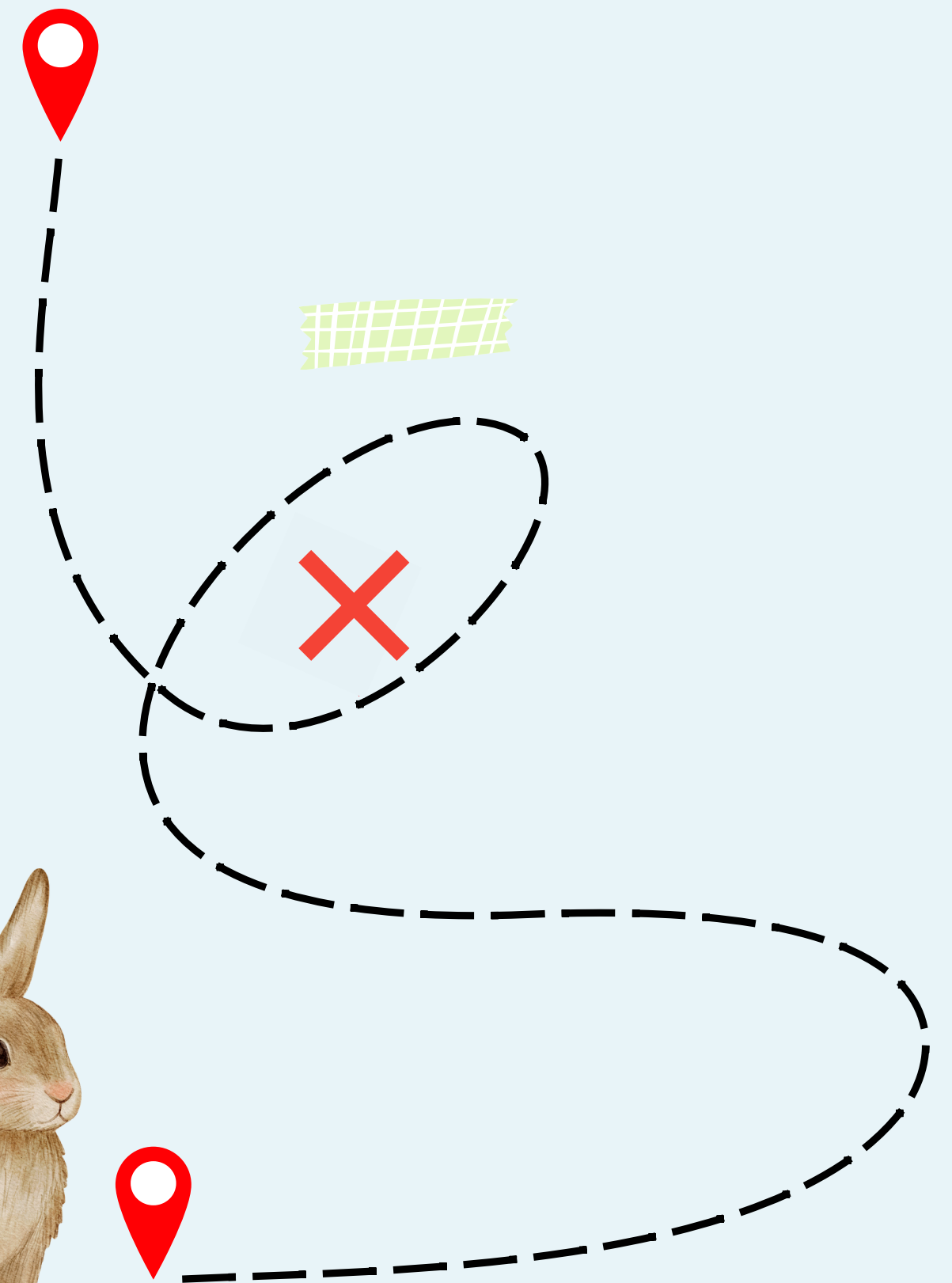
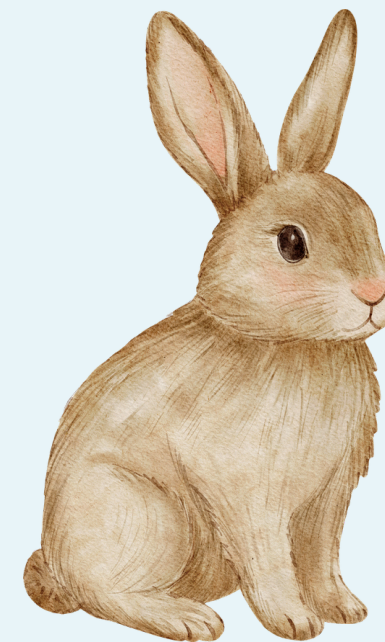
SYSTEM CONTEXT

- **Environment:** 8X8 Forest Grid
- **Entities:** 4 Species (Rabbit, Fox, Turtle, Deer)
- **Actions:** 2 per turn (Feed or Migrate)

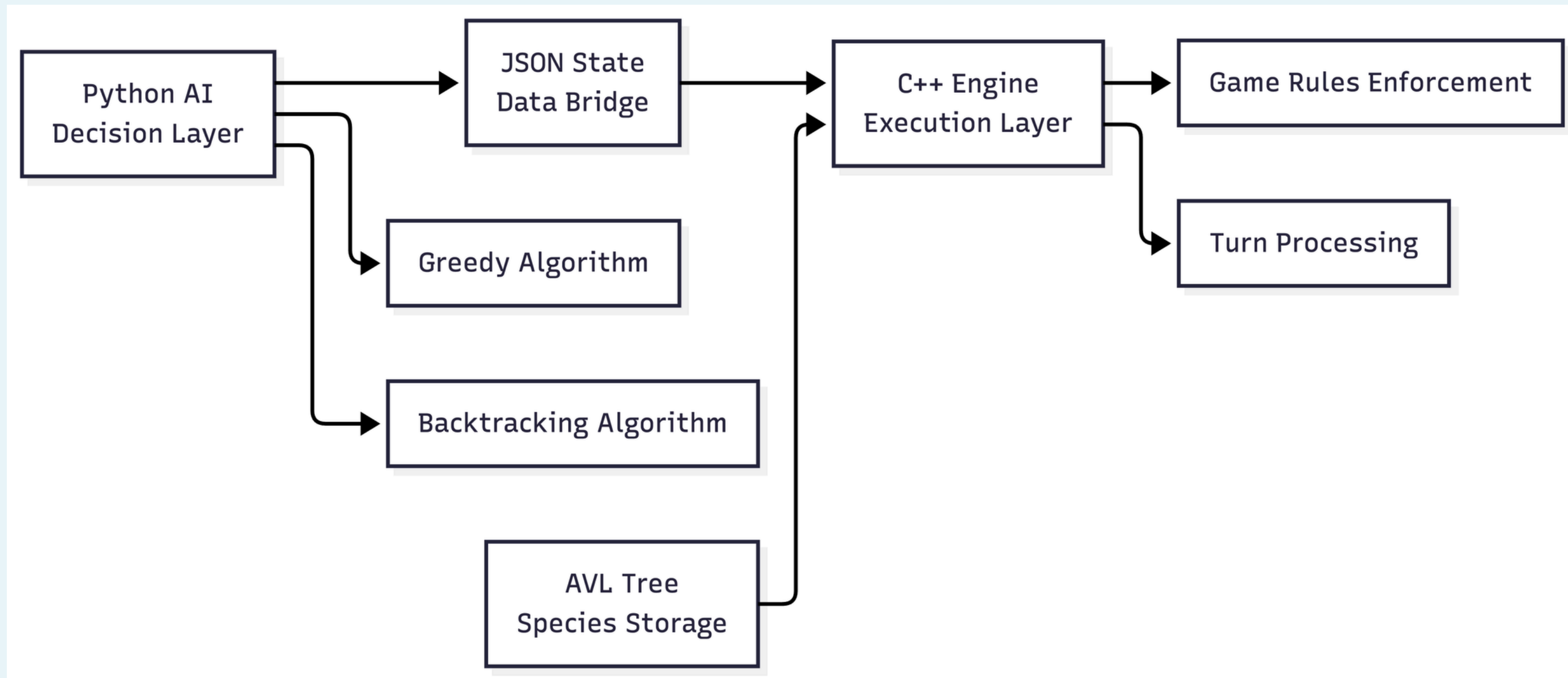


ALGORITHMIC ENGINE (LOGIC FLOW)

- **GREEDY ADVISOR: HIGH-RISK SPECIES RECOMMENDATION**
- **BACKTRACKING PATH: SAFE ROUTE CALCULATION**
- **FEEDBACK LOOP: ALGORITHMS DRIVE GAME DECISIONS**



ALGORITHMIC ENGINE (LOGIC FLOW)



GREEDY STRATEGY

THE GREEDY ALGORITHM FUNCTIONS AS A REAL-TIME SURVIVAL ADVISOR THAT PERFORMS LOCAL OPTIMIZATION TO MANAGE SPECIES VULNERABILITY. BY ANALYZING A HEALTH MATRIX THAT INCLUDES CURRENT HUNGER LEVELS AND POPULATION COUNTS



PSEUDOCODE

```
best ← null  
highestRisk ← -1
```

```
for each species:
```

```
  if species is active:
```

```
    risk ← hunger + (6 - population)
```

```
    if risk > highestRisk:
```

```
      highestRisk ← risk
```

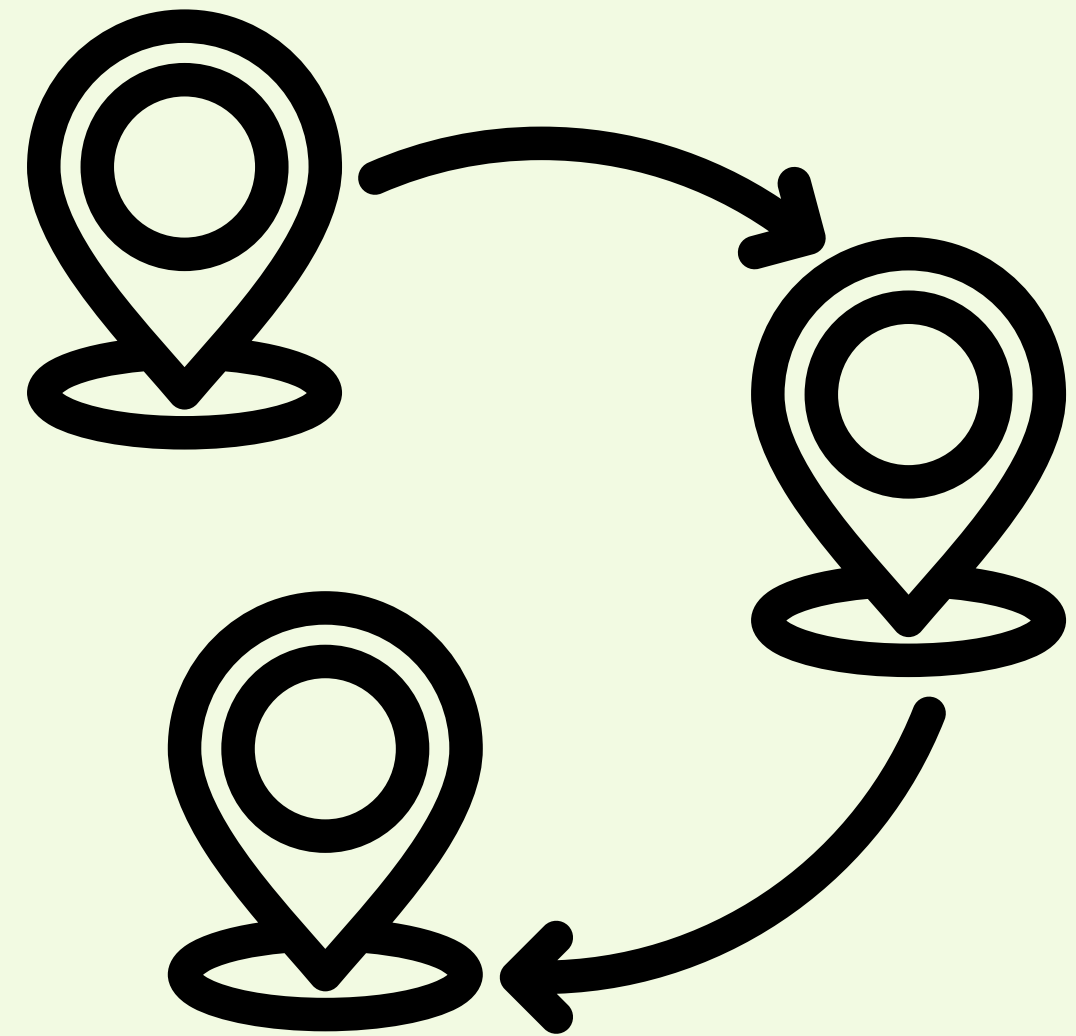
```
      best ← species
```

```
return best
```



BACKTRACKING: SAFE MIGRATION PATH

BACKTRACKING IS UTILIZED AS A RECURSIVE NAVIGATION ENGINE TO FIND SAFE MIGRATION PATHS TOWARD REFUGE ZONES, EXPLORING THE 8X8 GRID SIMULATING MOVEMENT WHILE AVOID PREDATORS AND OBSTACLES.



PSEUDOCODE

FUNCTION FIND_MIGRATION_PATH(board, start_row, start_col, symbol):

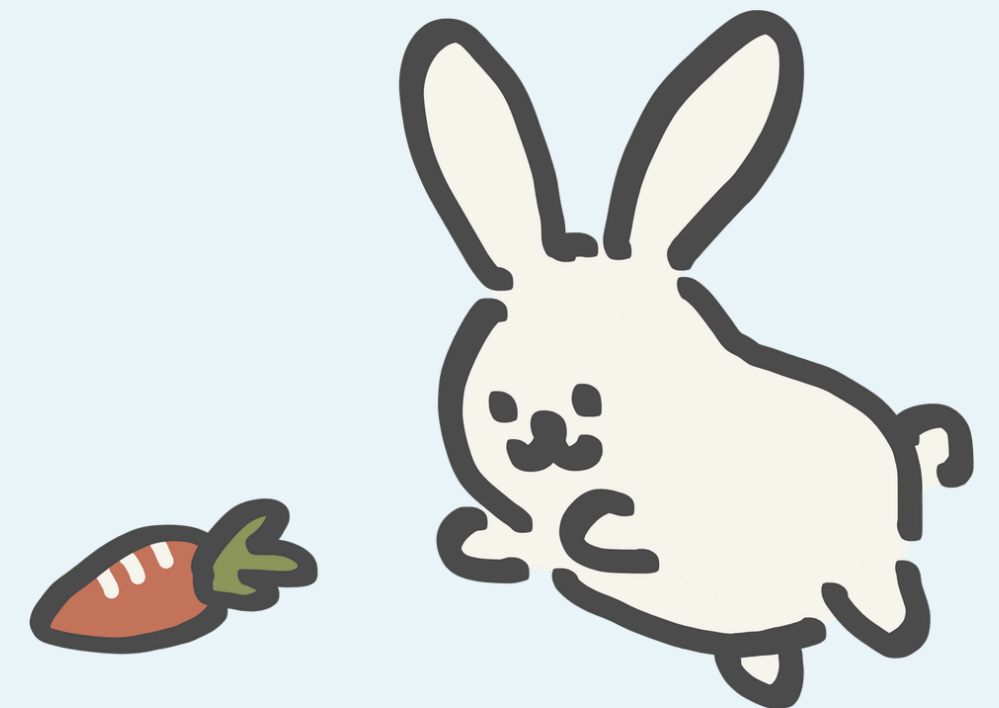
rows $\leftarrow$ number of rows in board
cols $\leftarrow$ number of columns in board

visited $\leftarrow$ matrix [rows][cols] initialized to False
path $\leftarrow$ empty list

add [start_row, start_col] to path

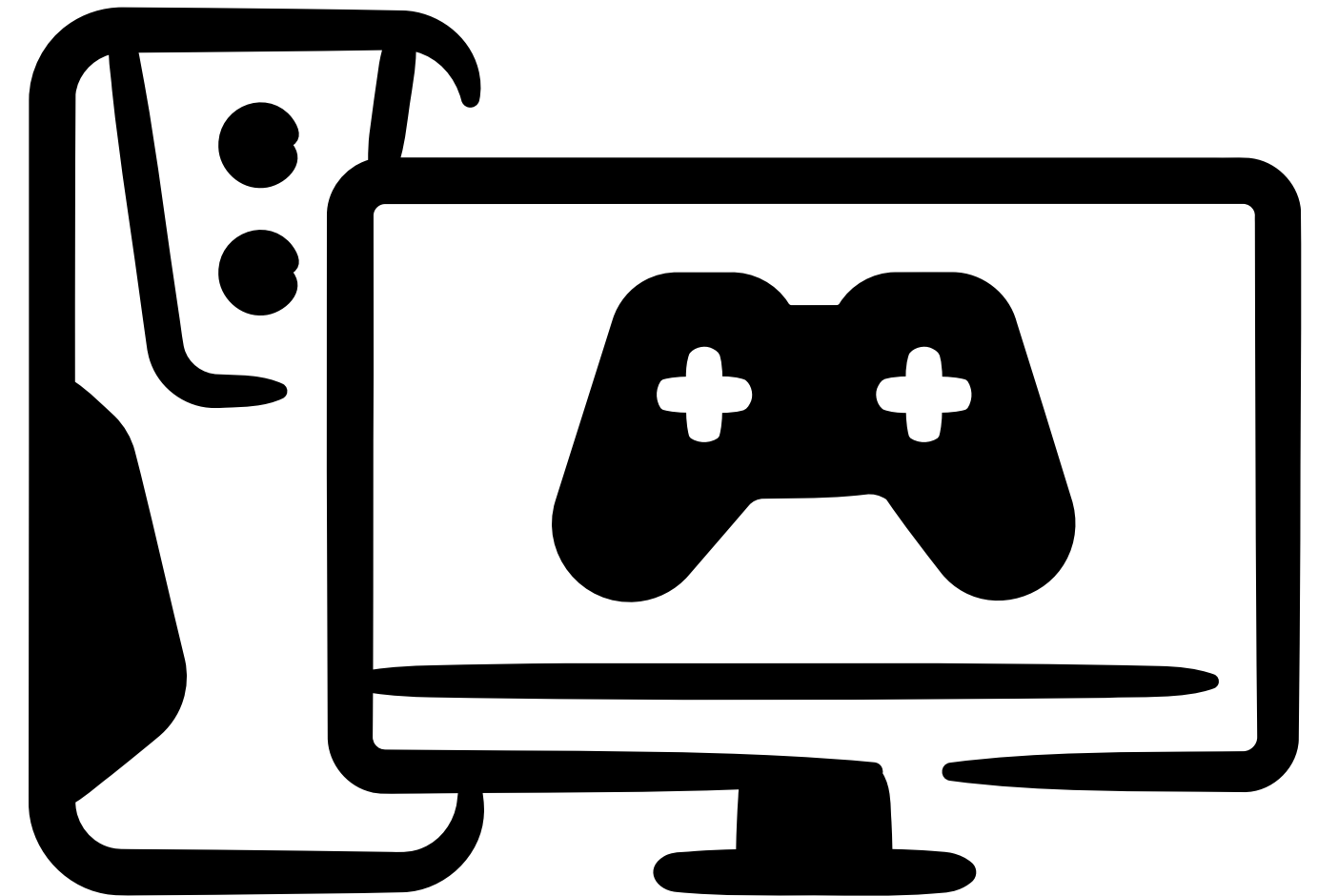
IF DFS(start_row, start_col, visited, path, symbol) IS TRUE:
 RETURN path

ELSE:
 RETURN empty list

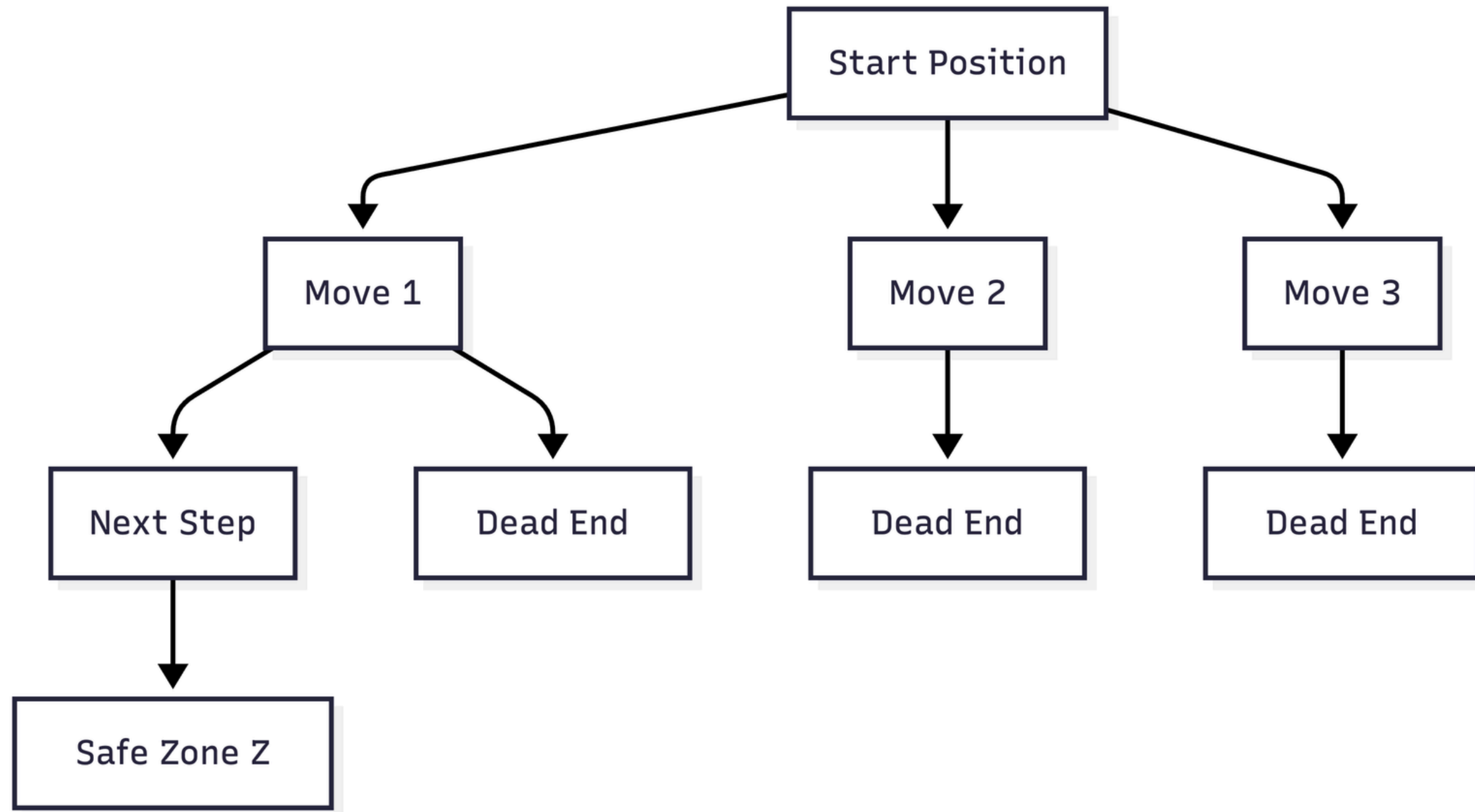


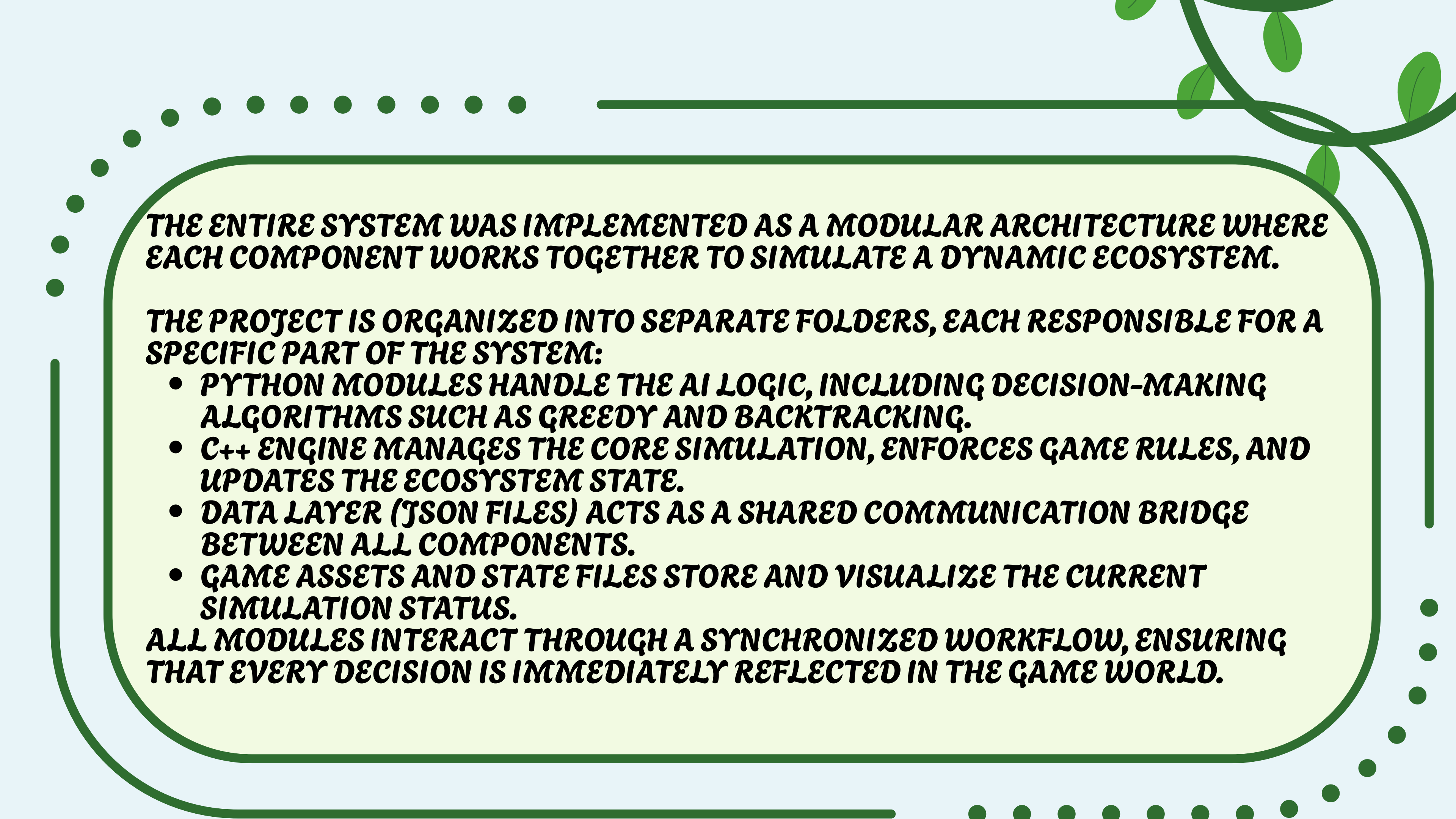
DIGITAL ECOSYSTEM

THE SYSTEM IS DESIGNED AS A MULTI-LAYER ARCHITECTURE. PYTHON HANDLES ALGORITHMIC DECISION-MAKING (GREEDY AND BACKTRACKING), WHILE C++ EXECUTES THE CORE SIMULATION LOGIC AND RULE ENFORCEMENT. A SHARED JSON STATE FILE ACTS AS AN INTERMEDIATE COMMUNICATION LAYER, SYNCHRONIZING BOTH MODULES AND MAINTAINING A CONSISTENT GAME STATE ACROSS EXECUTIONS.



BACKTRACKING TREE





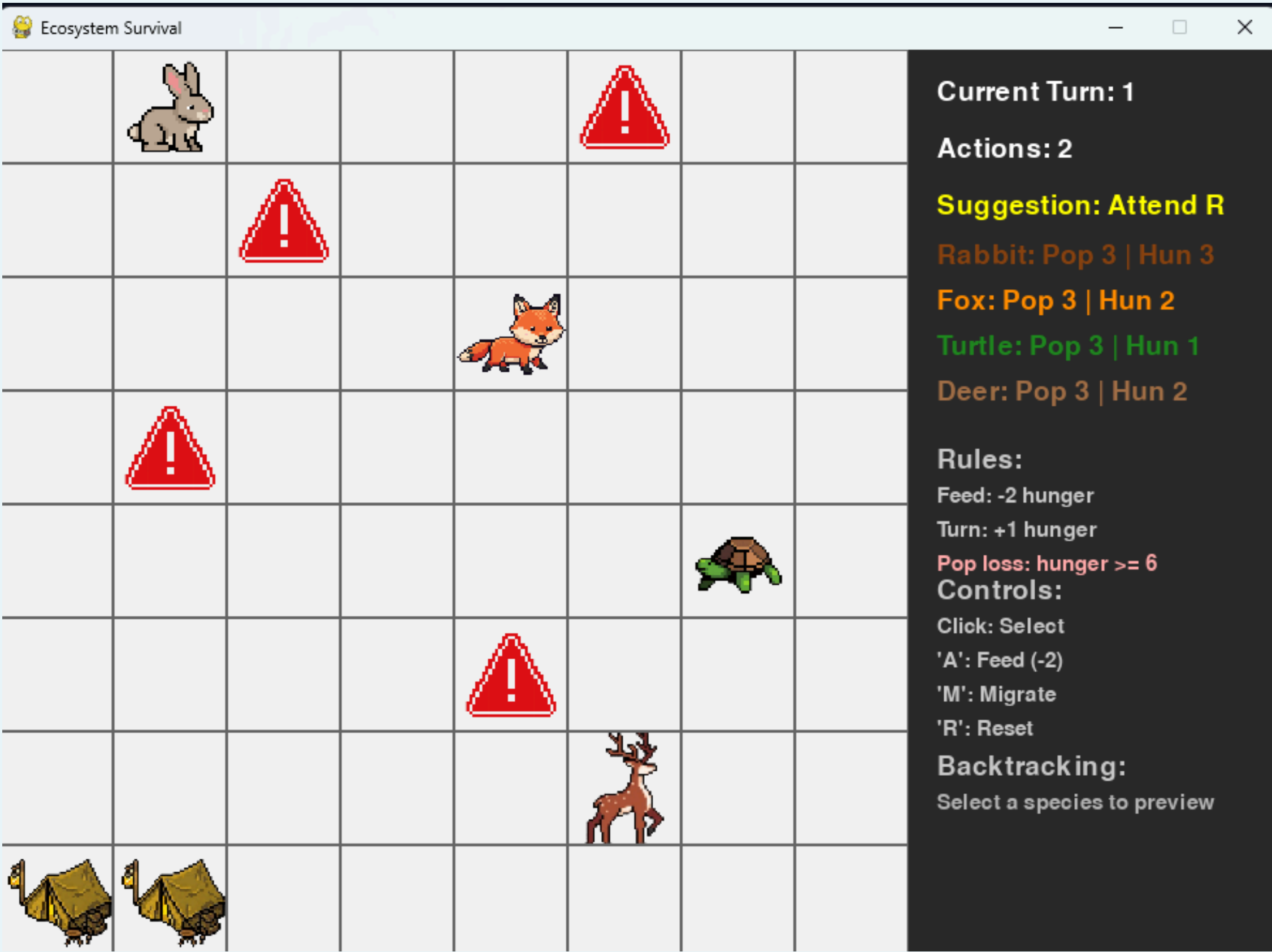
THE ENTIRE SYSTEM WAS IMPLEMENTED AS A MODULAR ARCHITECTURE WHERE EACH COMPONENT WORKS TOGETHER TO SIMULATE A DYNAMIC ECOSYSTEM.

THE PROJECT IS ORGANIZED INTO SEPARATE FOLDERS, EACH RESPONSIBLE FOR A SPECIFIC PART OF THE SYSTEM:

- **PYTHON MODULES HANDLE THE AI LOGIC, INCLUDING DECISION-MAKING ALGORITHMS SUCH AS GREEDY AND BACKTRACKING.**
- **C++ ENGINE MANAGES THE CORE SIMULATION, ENFORCES GAME RULES, AND UPDATES THE ECOSYSTEM STATE.**
- **DATA LAYER (JSON FILES) ACTS AS A SHARED COMMUNICATION BRIDGE BETWEEN ALL COMPONENTS.**
- **GAME ASSETS AND STATE FILES STORE AND VISUALIZE THE CURRENT SIMULATION STATUS.**

ALL MODULES INTERACT THROUGH A SYNCHRONIZED WORKFLOW, ENSURING THAT EVERY DECISION IS IMMEDIATELY REFLECTED IN THE GAME WORLD.

UI GAME LOOK



THANKS!

